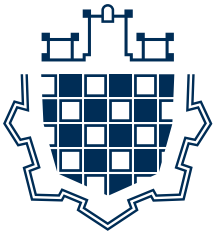

UNIVERSITY OF MANNHEIM

School of Business Informatics and Mathematics

Test Suite Minimisation in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Large-scale industrial software systems face significant challenges in executing test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [3]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case, test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [25]. Test-suite minimisation is particularly critical for regression testing, where previously passing tests are re-executed after code changes to detect newly introduced defects. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation in the context of the *Large-Scale Software Observatorium* (LASSO) [15], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence. Its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems (large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes [2]).

Our approach leverages *Stimulus–Response Matrices* (SRMs) [15], a two-dimensional representation mapping test stimuli to observed execution outcomes. We enrich SRM entries with per-test coverage information and utilise the SRM's existing

mutation data, enabling minimisation algorithms to operate on fine-grained behavioural information rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along six established criteria: SRM compatibility, coverage retention, reduction effectiveness, mutation-coverage preservation, computational scalability, and implementation feasibility. The Mutation-Aware Enhanced Harrold–Gupta–Soffa (MA-EHGS) algorithm emerged as the most promising candidate. MA-EHGS extends the empirically validated Enhanced HGS baseline [10] by integrating mutation weighting to balance coverage preservation and fault-detection power [13].

We successfully integrated MA-EHGS into LASSO and validated the implementation using SRM representations of JUnit-based test suites. The achieved reduction levels fall within the ranges reported for HGS-based and mutation-aware greedy techniques in prior empirical studies [10, 13]. Since MA-EHGS follows the same matrix-driven selection structure as other greedy test-suite minimisation algorithms, it retains the practical scalability widely observed for greedy heuristics in regression testing [25, 18].

Contents

List of Figures

List of Tables

List of Abbreviations

ABC	Artificial Bee Colony
AST	Abstract Syntax Tree
CFG	Control Flow Graph
CSP	Constraint Satisfaction Problem
DG	Delayed Greedy
EHGS	Enhanced HGS
GA	Genetic Algorithm
GE	Greedy Effective
GRE	Greedy Redundant Eliminating
HGS	Harrold–Gupta–Soffa algorithm
ILP	Integer Linear Programming
LASSO	Large-Scale Software Observatorium
LSL	LASSO Scripting Language
MA-EHGS	Mutation-Aware Enhanced HGS
MOEA	Many-Objective Evolutionary Algorithm
NEMO	Nonlinear Multi-criteria Integer Programming
NSGA-II	Non-dominated Sorting Genetic Algorithm II
PSO	Particle Swarm Optimisation
SAT	Boolean Satisfiability Problem
SRM	Stimulus–Response Matrix
SSR	Suite Size Reduction

1. Introduction

Software testing ensures that program modifications do not introduce defects. As systems evolve, test suites grow and increase execution cost and maintenance effort. Modern suites often combine human-written, automatically generated, and in recent years more frequently AI-generated tests. Tools such as Randoop [20] and EvoSuite [8] can produce thousands of tests, and large industrial suites may require days or weeks to execute [22]. Although such suites achieve high coverage and mutation scores, they frequently contain redundant tests whose requirements are subsumed by others, resulting in prolonged execution, increased costs, and elevated maintenance overhead.

Test-suite minimisation addresses this issue by systematically removing redundant tests while preserving fault-detection capability [25]. Techniques range from *coverage-preserving* approaches, which maintain all coverage requirements, to *coverage-relaxing* approaches, which accept limited coverage loss for stronger reduction. While minimisation can significantly reduce computational demand and feedback latency, overly aggressive reduction risks discarding tests with unique defect-revealing behaviour. Effective minimisation therefore requires balancing reduction efficiency with fault-detection retention.

1.1. Background and Motivation

Testing remains a major cost driver in software engineering. Classical studies estimate that verification and validation account for up to half of total development effort [4]. The cost pressure persists in modern continuous-integration environments, where automated or AI-generated suites may contain tens of thousands of tests [22, 3]. Executing all tests after each change is often infeasible, motivating techniques that reduce suite size while retaining quality. Test suite minimisation has been exploring this issue for decades, with numerous algorithms proposed to

identify and eliminate redundant tests [25], by leveraging coverage and quite recently mutation information as a proxy for fault-detection capability [13]. Details on coverage and mutation testing are provided in Chapter 2.

1.2. LASSO Context

The *Large-Scale Software Observatorium* (LASSO) [15] is a research platform for scalable analysis of large software corpora (*Big Code*). LASSO automates the extraction of static properties and dynamic execution behaviour from open-source systems and enables reproducible pipelines through the *LASSO Scripting Language* (LSL). This makes it a central infrastructure for empirical software engineering and for evaluating emerging techniques such as AI-assisted code generation.

LASSO represents system behaviour through *Stimulus–Response Matrices* (SRMs) [15], where rows denote test stimuli, columns denote systems, and cells contain structured response objects. SRMs allow language-independent extraction of coverage metrics, mutation scores, and behavioural characteristics, enabling systematic comparison of functionally equivalent implementations. Section 2.2 provides formal definitions.

Despite supporting large-scale behavioural and coverage analysis, LASSO currently lacks integrated test-suite minimisation. Redundant test executions accumulate significant computational overhead across the platform. Integrating minimisation would reduce this overhead while preserving the behavioural coverage required for reliable system analysis.

1.3. Research Objectives and Questions

This thesis investigates how test-suite minimisation can be integrated into LASSO using its SRM abstraction. The study is guided by:

1. **RQ1:** Which minimisation techniques demonstrate the highest practical effectiveness in the literature for the LASSO platform?
2. **RQ2:** Which evaluation criteria best suit minimisation within LASSO's SRM-based environment?

3. **RQ3:** Which approach best balances reduction effectiveness, coverage preservation, mutation preservation and scalability?
4. **RQ4:** How can the selected approach be integrated into LASSO while maintaining language independence?

1.4. Implementation Strategy

This thesis performs minimisation directly on SRMs rather than relying on external tools, which typically require source-code access or language-specific instrumentation. SRMs already encapsulate all data required for behavioural minimisation (test identifiers, execution results, coverage information, and mutation data), allowing language-independent and scalable processing while ensuring seamless integration into LASSO's analysis pipeline.

1.5. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces coverage, mutation testing, SRMs, and formally defines the minimisation problem.

Chapter 3 surveys greedy, exact, search-based, and data-driven minimisation techniques.

Chapter 4 defines evaluation criteria (C1–C6) and the scoring framework.

Chapter 5 applies the framework to select the Mutation-Aware Enhanced HGS approach.

Chapter 6 describes the implementation and integration into LASSO.

Chapter 7 discusses limitations, trade-offs, and deployment considerations.

Chapter 8 concludes and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this work. It introduces the core concepts of test coverage and mutation testing, explains the *Stimulus–Response Matrix* (SRM) representation used within LASSO, and formalises the test-suite minimisation problem, including trade-offs between coverage preservation and reduction strength.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements exercised by a test suite [26]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Coverage criteria define program "parts" at different granularity levels [26]. While numerous coverage types exist (e.g., instruction coverage, path coverage, class coverage), this work focuses on the three metrics most commonly reported in test-suite minimisation literature [25, 11, 13, 18]:

- **Line Coverage:** proportion of source lines executed.
- **Branch Coverage:** proportion of conditional branches whose true/false outcomes are both executed.
- **Method Coverage:** proportion of methods invoked at least once.

These three metrics capture execution at different granularities and are widely supported by standard coverage tools such as JaCoCo. Combining multiple metrics provides a more complete assessment of test quality, as high line coverage with low branch coverage may miss conditional faults.

2.1.2. Mutation Score

Mutation testing measures test effectiveness by introducing small syntactic changes (*mutants*) into the program and observing whether tests detect the injected faults [14]. The *mutation score* is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

A mutant is *killed* if the test causes observable behavioural divergence from the original program.

Mutation Types. Two principal mutation styles are distinguished [19]:

1. **Weak Mutation:** detects state deviations immediately after a mutated statement executes.
2. **Strong Mutation:** detects externally visible behavioural differences at program output.

Weak mutation captures local sensitivity to code changes. Strong mutation quantifies global behavioural divergence. Mutation scores thereby approximate a suite's fault-detection capability.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute LASSO's central data structure for *Test-Driven Software Experimentation* [15]. They provide a unified, language-independent format for recording and comparing run-time behaviour under controlled stimuli.

2.2.1. Sequence and Actuation Sheets

Sequence Sheets describe ordered stimuli applied to systems under test. Each row represents one method invocation with inputs and expected outputs. **Actuation Sheets** record the corresponding observed outputs and other run-time data during execution.

Stimulus (Sequence Sheet)

Out	Operation	Service	Input
create	`create`	`'Stack`	
–	`push`	A1	42
42	`pop`	A1	

Observed Behaviour (Actuation Sheet)

Out	Operation	Service	Input
`<instance>`	`create`	`'Stack`	
`true`	`push`	A1	42
`42`	`pop`	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to SRMs

LASSO generalises stimulus-to-actuation mappings into multidimensional *Stimulus–Response Matrices* (SRMs):

$$M = [M_{ij}], \qquad M_{ij} = \text{Actuation}(t_i, v_j),$$

where each row t_i represents a stimulus (test), each column v_j a system variant, and each cell M_{ij} stores execution outcomes and associated analysis data such as coverage and mutation attributes.

	System 1	System 2	System 3	System 4
Test 1	42	41	42	43
Test 2	100	100	100	100
Test 3	true	true	false	true

Figure 2.2.: Example SRM fragment showing actual outputs across different system variants (columns). Green cells indicate passing tests; red cells indicate failures. Systems 1 and 3 produce identical output (42) for Test 1, while Systems 2 and 4 produce different mutant outputs (41, 43). All systems pass Test 2 with identical output (100).

SRMs unify functional results, coverage data, and mutation outcomes in a single matrix, enabling language-independent behavioural analysis.

2.3. The Test-Suite Minimisation Problem

2.3.1. Concept and Objectives

Test-suite minimisation seeks to remove redundant tests from an existing suite while maintaining sufficient coverage and fault-detection capability [25]. Given a suite $T = \{t_1, \ldots, t_n\}$ and a set of coverage requirements $R = \{r_1, \ldots, r_m\}$, each test t_i covers a subset $C(t_i) \subseteq R$. The objective is to find a smaller subset $S^* \subseteq T$ that provides comparable or identical coverage.

Two major formulations exist:

- **Coverage-preserving minimisation:** guarantees full coverage, $C(S^*) = C(T)$. It ensures no requirement is lost but often limits reduction potential.
- **Coverage-relaxing minimisation:** allows limited loss of coverage, $C(S^*) \subset C(T)$, to achieve greater reduction. This variant trades fault-detection certainty for smaller suite size.

Both forms address the same optimisation tension: *minimise suite size while retaining sufficient adequacy.* The general optimisation can be expressed as

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad |C(S)| \geq \gamma \cdot |C(T)|,$$

where $0 < \gamma \leq 1$ controls the acceptable coverage retention threshold ($\gamma = 1$ for coverage-preserving, $\gamma < 1$ for coverage-relaxing).

2.3.2. Computational Complexity and Trade-offs

The coverage-preserving variant corresponds to the *minimum set-cover problem*, which is NP-complete [25]. No polynomial-time algorithm guarantees optimality for arbitrary inputs, and practical implementations rely on heuristics or approximations [11, 6]. Some greedy heuristics achieve linear complexity $O(nm)$ in practice, balancing scalability and quality.

Coverage-relaxing algorithms may yield stronger reductions but risk discarding unique defect-revealing tests, potentially lowering mutation scores or missing rare execution paths. Hence, every minimisation approach must balance the trade-off between computational efficiency, reduction rate, and fault-detection retention.

Original Test Suite

$T = \{t_1, \ldots, t_n\}$

$|T| = n$ tests

$C(T) = \bigcup_{i=1}^{n} C(t_i)$

Complete coverage

Reduced Test Suite

$S^* \subseteq T$

$|S^*| < n$ tests

$C(S^*) = C(T)$ (preserving)

or $C(S^*) \subset C(T)$ (relaxing)

Optimisation Goal

Minimise: $|S^*|$

Subject to: $C(S^*) \geq \gamma \cdot C(T)$, where $0 < \gamma \leq 1$

Complexity: NP-complete (set-cover equivalent)

Figure 2.3.: The test-suite minimisation problem formulated as an optimisation balancing suite size and coverage retention.

2.3.3. Minimisation within LASSO

In LASSO, each test corresponds to one SRM row. Coverage and mutation data stored in SRM cells provide the requirement mappings $C(t_i)$. Minimisation thus operates directly on SRMs by selecting a subset of rows S^* that preserves coverage and mutation adequacy across all variants:

$$C(S^*)_{\text{SRM}} \geq \gamma \cdot C(T)_{\text{SRM}}.$$

This SRM-based formulation supports both coverage-preserving and coverage-relaxing strategies while remaining language-independent, enabling scalable experimentation across diverse software systems.

2.3.4. Algorithmic Categories

Following established classifications [25, 21, 3], algorithms group into four categories: **greedy heuristics** (classical greedy, HGS [11]), **exact optimisation** (ILP, constraint-satisfaction [17]), **search-based methods** (genetic algorithms [25]), **data-driven methods** (clustering, overlap-aware [3]). Detailed explanation in Chapter 3.

3. Literature Review

This chapter surveys test-suite minimisation algorithms across four main families: greedy heuristics, exact optimisation approaches, search-based (metaheuristic) techniques, and data-driven similarity-based methods. The focus is on how these approaches formulate the minimisation problem, which information they use (e.g. coverage matrices, mutants, control-flow graphs), and how they construct reduced test suites. Comparative scoring and ranking are deferred to Chapter 5.

3.1. Greedy Heuristics

Greedy heuristics are among the most widely studied techniques for test-suite minimisation [25, 18]. They operate directly on a test–requirement incidence matrix (often a coverage matrix) and construct a reduced suite incrementally: in each step, the algorithm selects a test that contributes beneficial coverage for currently uncovered requirements, until all requirements are covered. Because of their simplicity, low implementation overhead, and good practical performance, greedy algorithms are frequently used as baselines in empirical studies.

3.1.1. Classical Greedy

The classical greedy heuristic repeatedly selects the test that covers the largest number of yet-uncovered requirements. Chvátal showed that this algorithm achieves a logarithmic approximation bound for the set-cover problem [6]. Rothermel et al. and later surveys treat classical greedy as a standard baseline for regression test-suite minimisation [22, 25].

In a typical formulation, the algorithm maintains a set of uncovered requirements and, in each iteration, scores every remaining test by the number of uncovered requirements it satisfies. The test with the highest score is added to the reduced suite, and its covered requirements are marked as covered. This continues until no

uncovered requirements remain. The algorithm operates purely on the coverage matrix, without additional program structure.

Noemmer and Haas observe that such greedy variants are "the test suite minimization algorithms most commonly found in research" [18]. In their Java study, they investigate both classical greedy and more advanced greedy heuristics with respect to structural coverage and mutation testing. Jehan and Wotawa report similar results for JavaScript projects and use classical greedy and related variants as central baselines in their empirical analysis [13].

3.1.2. Harrold–Gupta–Soffa (HGS)

Harrold, Gupta and Soffa proposed the Harrold–Gupta–Soffa (HGS) algorithm as one of the early coverage-driven minimisation heuristics [11]. HGS extends the basic greedy idea by taking into account the "difficulty" of covering certain requirements. Roughly, the algorithm proceeds in phases:

- Tests are grouped according to the requirements they cover.
- Requirements that are covered by only a few tests are given priority.
- At each step, HGS selects tests that help cover such "hard-to-cover" requirements while avoiding redundant tests whose coverage is already subsumed by others.

Harrold et al. derive a worst-case time bound of $O(n(n+m)MAXCARD)$, where n denotes the number of testing sets, m the number of test cases, and $MAXCARD$ the maximum number of requirements covered by any single test [11]. HGS has since become a standard greedy baseline and appears in many comparative studies of test-suite minimisation [25, 5, 18].

Chen and Lau introduce two related greedy variants, Greedy Effective (GE) and Greedy Redundant Eliminating (GRE), which modify how tests are selected and how redundant tests are removed, but still operate on the same coverage-matrix abstraction [5].

3.1.3. Delayed Greedy (DG)

Tallam and Gupta propose the Delayed Greedy (DG) algorithm to address a limitation of classical greedy and HGS: if very large tests are selected early, they

may dominate the solution and render other tests redundant, even when those smaller tests might later have formed part of a more compact suite [23].

DG uses formal concept analysis to identify and remove subsumed tests before the greedy selection phase. Conceptually, the algorithm:

- Builds a concept lattice over tests and requirements, where each concept represents a set of tests with a common set of covered requirements;
- Eliminates tests whose coverage is entirely contained in the coverage of other tests or combinations of tests;
- Applies a greedy selection strategy on the reduced set of tests.

This two-step design aims to reduce redundancy before greedy selection, potentially leading to smaller resulting suites. DG still operates on the test–requirement matrix, but the concept lattice construction incurs additional computational and implementation overhead compared with simpler greedy methods [23].

3.1.4. Enhanced HGS (EHGS)

Gladston and Ramesh refine HGS and evaluate several variants on Java programs, introducing the *Enhanced HGS* (EHGS) algorithm [10]. EHGS retains the basic HGS structure but emphasises two ideas:

1. **Unique coverage first:** EHGS first selects tests that are the only ones to cover certain requirements (so-called essential tests).
2. **Residual coverage next:** In subsequent steps, EHGS ranks remaining tests by how many still-uncovered requirements they satisfy and selects tests in decreasing order of this residual coverage.

The first phase ensures that no requirement uniquely associated with a single test is accidentally lost. The second phase then applies a coverage-based greedy strategy restricted to the remaining requirements. Gladston and Ramesh evaluate EHGS and related variants empirically and show that EHGS yields competitive suite sizes in their benchmarks [10].

EHGS operates entirely on a test–requirement matrix and does not depend on language-specific control-flow information. This makes it a natural candidate for SRM-based settings where coverage data are available in matrix form.

3.1.5. Mutation-Aware Enhanced HGS (MA-EHGS)

The Mutation-Aware Enhanced HGS (MA-EHGS) algorithm, proposed in this thesis, extends EHGS to explicitly incorporate mutation information. Recent empirical studies have shown that purely coverage-based minimisation can reduce mutation score, especially when reduction is aggressive [18, 13]. At the same time, many-objective evolutionary algorithms demonstrate that treating mutation score as an explicit optimisation objective can help preserve fault detection while still reducing cost [24]. This evolution in the literature reflects a broader shift from purely coverage-driven adequacy towards fault-based adequacy metrics.

MA-EHGS builds on this development by embedding mutation awareness into a deterministic, greedy, SRM-based heuristic. Conceptually, it operates as follows:

Requirements-Matrix Formulation

MA-EHGS uses a binary matrix $M \in \{0,1\}^{n \times m}$ where each row corresponds to a test and each column to a requirement. Structural coverage requirements (e.g. lines, branches, methods) and mutation requirements (mutants killed by a test) are represented uniformly as columns in M. For each test t_i, the set of requirements it satisfies is given by the non-zero entries in row i.

Two-Phase Selection with Mutation Weighting

MA-EHGS follows the two-phase structure of EHGS:

Phase 1: Identify and select tests that uniquely cover at least one requirement (structural or mutant). These tests are essential, because their removal would leave some requirement uncovered.

Phase 2: For the remaining uncovered requirements, repeatedly select the next test using a weighted score

$$\text{score}(t_i) = \alpha \cdot |\text{newCoverage}_i| + \beta \cdot |\text{newMutants}_i|,$$

where newCoverage_i and newMutants_i denote, respectively, the sets of yet-uncovered structural and mutation requirements covered by t_i, and $\alpha, \beta \in$

$[0, 1]$ control the relative importance of structural versus mutation requirements.

When $\beta = 0$, MA-EHGS reduces to a coverage-only variant similar to EHGS. For $\beta > 0$, the selection process explicitly prioritises tests that kill additional mutants. The algorithm thus bridges two strands of the literature: greedy coverage-based minimisation [25, 18] and mutation-aware optimisation [13, 24].

Complexity Considerations

MA-EHGS retains the data structures and overall complexity of EHGS. Phase 1 scans the matrix to find essential tests; Phase 2 iteratively computes scores based on the remaining uncovered requirements. Both phases operate on the SRM representation and do not require additional program-structure analyses. Therefore MA-EHGS is designed to preserve the scalability characteristics of EHGS while extending it with mutation awareness.

3.2. Exact Optimisation Approaches

Exact optimisation approaches treat test-suite minimisation as a formal optimisation problem, typically formulated as Integer Linear Programming (ILP), Boolean Satisfiability (SAT), or related constraint models. These methods aim to compute an optimal solution according to a given objective, but inherit the NP-hardness of the underlying hitting-set formulation [9].

3.2.1. ILP, SAT, and Set-Cover Formulations

In ILP formulations, each test is associated with a binary decision variable indicating whether it is selected, and each requirement induces a constraint ensuring that at least one selected test covers it [25, 7]. The objective function typically minimises the number of selected tests or a weighted cost reflecting test execution time.

SAT and Constraint Satisfaction Problem (CSP) encodings follow a similar idea, but express constraints in propositional logic or more general constraint languages [12]. The resulting formulas are then solved using off-the-shelf SAT or

CSP solvers. These formulations provide a precise link between test-suite minimisation and classical optimisation technology.

While worst-case complexity remains exponential, ILP and SAT can solve small- to medium-sized instances and are often used to provide optimality baselines for evaluating heuristics [25, 21].

3.2.2. Nonlinear Multi-Criteria Optimisation (NEMO)

Lin et al. argue that linear models may not capture interactions between test cases, especially when multiple criteria such as cost and fault detection need to be considered simultaneously [16]. Their Nonlinear Multi-criteria Integer Programming (NEMO) framework extends the linear ILP formulation with nonlinear terms to model such interdependencies.

NEMO allows multiple objectives, such as maximising fault detection while minimising execution cost, and encodes them as a single nonlinear integer program. This approach demonstrates how richer optimisation models can be used to express complex adequacy notions. However, the increased model expressiveness comes at the price of higher computational effort compared with linear ILP [16].

3.2.3. Structure-Aware Optimisation (REDUNET)

Mongiovi et al. propose REDUNET, which combines set-cover reasoning with optimisation over Control Flow Graphs (CFGs) [17]. Instead of working solely with a coverage matrix, REDUNET analyses control-flow and data-flow dependencies in the program to identify redundant tests. It then uses ILP-based reasoning over this CFG-enriched representation to select a reduced suite.

REDUNET illustrates a different dimension in the design space: leveraging program structure (CFGs) to guide test reduction. This approach can exploit structural information beyond what is available in a pure SRM setting, but it also requires access to language-specific program representations and associated analysis tools.

3.3. Search-Based and Metaheuristic Approaches

Search-based approaches treat test-suite minimisation as an optimisation problem and use metaheuristics such as Genetic Algorithms (GAs) or swarm-intelligence methods to explore the space of candidate subsets [25, 21]. Instead of constructing a single solution greedily, these techniques maintain a population of candidate solutions and iteratively improve them via stochastic operators.

3.3.1. Genetic Algorithms

Genetic Algorithms represent each candidate test suite as a chromosome, often a binary vector indicating which tests are selected [25]. A typical GA for test-suite minimisation proceeds as follows:

- Initialise a population of random or heuristically constructed test subsets;
- Evaluate each candidate using a fitness function, such as coverage achieved minus cost, or a multi-objective fitness vector;
- Select fitter candidates for reproduction;
- Apply crossover (combining parts of two parents) and mutation (randomly flipping selection bits) to generate new candidates;
- Repeat evaluation and selection over multiple generations.

GAs can accommodate complex objectives and constraints, but require choices for population size, number of generations, crossover and mutation rates, and other parameters [25, 21]. Many GA-based minimisation approaches still optimise primarily for coverage and cost, with mutation information incorporated less frequently [18].

3.3.2. Other Metaheuristic Approaches

Beyond GAs, several other metaheuristics have been applied to test-suite minimisation. Ahuja and Bhatia evaluate Particle Swarm Optimisation (PSO) and Artificial Bee Colony (ABC) algorithms, where each particle or bee represents a candidate suite and moves through the search space guided by heuristic rules [1].

These methods follow a similar pattern: they maintain a population of candidate solutions and iteratively update them using algorithm-specific operators.

Wei et al. explore Many-Objective Evolutionary Algorithms (MOEAs) such as Non-dominated Sorting Genetic Algorithm II (NSGA-II), MOEA/D, IBEA, and SPEA2-SDE, with multiple objectives including mutation score [24]. By treating mutation score as one objective among several, MOEAs can generate trade-off fronts between cost, coverage, and mutation preservation. This line of work demonstrates that mutation-aware optimisation is feasible within population-based search and provides additional evidence that mutation score is an important adequacy dimension alongside structural coverage.

3.4. Data-Driven and Similarity-Based Approaches

Data-driven and similarity-based approaches use information about coverage patterns to identify redundant or highly overlapping tests [25, 21]. Instead of solving a hitting-set formulation directly, they analyse the structure of coverage vectors and cluster or prioritise tests based on similarity.

3.4.1. Similarity-Based and Time-Constrained Methods

Similarity-based methods typically represent each test as a high-dimensional coverage vector and apply distance or similarity measures (e.g. Jaccard similarity) to identify groups of tests that cover similar requirements. Tests within the same cluster are candidates for reduction or reordering.

Bach et al. apply such techniques in a large industrial environment at SAP, where test execution is subject to strict time budgets [3]. Their methods penalise tests whose coverage overlaps strongly with other tests and prioritise those that provide new or diverse coverage. The goal is to maximise coverage within a given time budget rather than to compute a minimal hitting set.

Khan et al. survey similarity- and clustering-based methods and emphasise their flexibility for prioritisation and selection tasks [21]. However, because these methods focus on coverage overlap and time constraints, they are typically used for test-case prioritisation and selection under resource limits rather than for exact minimisation in the classical sense.

3.5. Summary and Motivation for Mutation-Aware Greedy Minimisation

The reviewed literature reveals several recurring themes and an evolution in how test-suite minimisation is approached:

- **Greedy heuristics as widely used baselines.** Classical Greedy, HGS and DG are consistently employed across empirical studies and are often used as reference algorithms for regression-test minimisation [25, 18, 13]. EHGS builds on this line of work by refining the selection strategy in a two-phase framework [10].
- **Exact methods and their limitations.** ILP, SAT and nonlinear models (e.g. NEMO) provide strong optimality baselines and can express sophisticated objectives, but become computationally expensive as the number of tests and requirements grows, reflecting the NP-complete nature of the underlying hitting-set problem [9, 25, 16, 18]. Structure-aware approaches such as REDUNET show how additional program information can be incorporated at the cost of language- and tool-specific dependencies [17].
- **Search-based and data-driven trade-offs.** Metaheuristics such as GAs, PSO, ABC and MOEAs have been used to model multiple objectives and constraints in a flexible way, but they usually require parameter tuning and entail higher computational cost [25, 21, 1, 24]. Similarity-based and overlap-aware methods exploit coverage patterns to prioritise tests under time budgets and are therefore more commonly applied to test-case prioritisation than to strict minimisation [3, 21].
- **Coverage–mutation gap and the rise of mutation-aware techniques.** Empirical studies show that coverage-only minimisation can reduce mutation scores, especially under aggressive reduction settings [18, 13]. At the same time, mutation-aware MOEAs demonstrate that explicitly taking mutation score into account, alongside cost and coverage, helps preserve fault-detection capability [24]. This suggests that structural coverage and mutation score capture complementary aspects of test-suite adequacy.
- **Motivation for mutation-aware greedy minimisation.** Against this background, it is natural to evolve existing coverage-based greedy algo-

rithms into mutation-aware variants. MA-EHGS follows this trend by extending the empirically studied EHGS with explicit mutation information while retaining the overall greedy, matrix-based structure. It combines the simplicity and established role of greedy minimisation with the fault-based perspective emerging from recent mutation studies.

The algorithms and families surveyed in this chapter form the basis for the comparative analysis in Chapter 5, where they are evaluated systematically against the criteria defined in Chapter 4.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines six evaluation criteria (C1–C6): one mandatory precondition and five performance dimensions, together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C6)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) The algorithm must operate directly on *Stimulus–Response Matrix* (SRM) representations without requiring control-flow graphs, syntax trees, or language-specific structural analysis [15]. Algorithms failing C1 receive a score of zero and are excluded. This precondition guarantees language-independent execution across heterogeneous software systems.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural coverage (instruction, line, branch) is retained after minimisation [26]. High retention indicates that the reduced suite maintains structural adequacy.

C3: Reduction Effectiveness Quantifies the proportion of test cases eliminated while preserving required coverage [25]. A high reduction rate reflects strong elimination of redundant tests and tangible computational savings.

C4: Mutation-Coverage Preservation Assesses fault-detection capability via mutation testing [13, 18]. The mutation score (ratio of killed mutants to total

mutants) acts as a fault-based adequacy metric complementing structural coverage.

C5: Computational Scalability Examines algorithm performance on large SRMs as the number of tests and requirements increases. Methods with near-linear complexity and efficient runtime behaviour are favoured [15, 3].

C6: Implementation Feasibility Considers practical integration factors such as implementation effort, parameter tuning, dependency overhead, and compatibility with LASSO's architecture [25]. Algorithms imposing heavy maintenance or configuration costs are penalised.

4.1.3. Justification of Criteria

The six criteria align directly with the research questions from Section 1.3, ensuring coherence between theoretical goals and empirical assessment.

C1 (SRM Compatibility) is mandatory because LASSO operates entirely on SRM abstractions [15]. Language independence requires that algorithms process abstract requirement matrices rather than language-specific artefacts.

C2–C4 (Quality Preservation) address adequacy and fault detection. **C2** ensures structural completeness (coverage retention). **C3** quantifies the degree of reduction. **C4** measures mutation-score retention, a strong empirical proxy for test effectiveness and fault detection capability [13]. Together, C2 and C4 provide comprehensive assurance that the minimised test suite maintains both structural coverage and fault-revealing capability.

C5 (Computational Scalability) ensures the algorithm remains feasible for large-scale use cases where SRMs may contain thousands of rows and columns [15, 3].

C6 (Implementation Feasibility) recognises the cost of practical adoption. Even theoretically optimal algorithms may become impractical if they require complex configuration or structural changes to the LASSO pipeline [25].

4.2. Ranking and Scoring Model

To enable systematic comparison, each algorithm is scored on a scale from 0 to 5 for each performance criterion (C2–C6). The scoring rubric is as follows:

- **Score 5:** Excellent performance; meets or exceeds best-known benchmarks in literature.
- **Score 4:** Good performance; achieves strong results with minor limitations.
- **Score 3:** Acceptable performance; meets basic requirements but with notable trade-offs.
- **Score 2:** Poor performance; significant shortcomings limit practical utility.
- **Score 1:** Very poor performance; fails to meet essential criteria.
- **Score 0:** Not applicable; fails mandatory precondition (C1).

After scoring, each algorithm's total score is computed as a sum with equal weighting across criteria except C1 (which is mandatory):

$$\text{Score}(A) = \begin{cases} 0, & \text{if } s_1(A) = 0 \text{ (fails C1)}, \\ \sum_{i=2}^{6} w_i \cdot s_i(A), & \text{otherwise.} \end{cases} \tag{4.1}$$

This scoring framework facilitates quantitative comparison across diverse algorithms, enabling informed selection based on empirical performance aligned with LASSO's requirements. The next chapter applies this framework to evaluate candidate minimisation techniques from the literature, ultimately selecting the most suitable approach for integration into LASSO.

5. Comparative Evaluation and Selection

This chapter evaluates candidate test-suite minimisation techniques against the criteria defined in Chapter 4 and justifies the selection of the Mutation-Aware Enhanced Harrold–Gupta–Soffa (MA-EHGS) algorithm for integration into LASSO. The focus is on how well different families of algorithms satisfy LASSO's requirements for Stimulus–Response Matrix (SRM) based analysis; detailed numerical scores and tables are provided in Appendix 8.3.

The evaluation covers the following four families (introduced in Chapter 3):

- **Greedy heuristics:** Classical Greedy, HGS, Enhanced HGS (EHGS), Delayed Greedy (DG), GE/GRE, and mutation-based greedy variants.
- **Exact and structural optimisation:** Integer Linear Programming (ILP), Boolean Satisfiability (SAT) and nonlinear formulations such as NEMO, plus structure-aware variants such as REDUNET.
- **Search-based and metaheuristic approaches:** Genetic Algorithms (GAs), Many-Objective Evolutionary Algorithms (MOEAs), including mutation-aware variants.
- **Data-driven and similarity-based approaches:** methods exploiting coverage similarity or overlap, often under time or resource constraints.

All techniques are assessed using criteria C1–C6: SRM compatibility, coverage retention, reduction effectiveness, mutation-score preservation, scalability, and implementation feasibility.

5.1. Criterion C1: SRM Compatibility

C1 requires that algorithms operate directly on SRMs without relying on control-flow graphs, abstract syntax trees, or other language-specific program structures.

Greedy heuristics (Classical Greedy, HGS, EHGS, DG) work on test–requirement incidence matrices [11, 5, 25, 18]. ILP, SAT and NEMO encode set cover over tests and requirements [22, 25, 7, 16]. Search-based and similarity-based methods represent candidate suites via coverage and mutation vectors [25, 21, 24, 3]. All of these are compatible with LASSO's SRM abstraction.

REDUNET, in contrast, requires control-flow graphs and program-structure analysis [17], which are not available in SRM-only settings. It therefore fails C1 and is excluded from further consideration.

Conclusion (C1). All matrix-based techniques satisfy C1; REDUNET does not and is excluded.

5.2. Criterion C2: Coverage Preservation

C2 measures whether an algorithm can retain structural coverage (lines, branches, methods) after minimisation.

Greedy heuristics guarantee 100% coverage by construction: they iteratively select tests until all requirements are covered [11, 5, 25, 18]. EHGS and DG preserve this property while refining the selection strategy [23, 10]. MA-EHGS inherits this guarantee and, in this thesis, treats both structural and mutation requirements as mandatory columns in the SRM.

Exact ILP/SAT formulations and NEMO enforce coverage constraints as hard constraints in their optimisation models [7, 25, 16]. Search-based and similarity-based approaches *can* preserve coverage when configured with hard constraints, but many variants treat coverage as one objective among several and may trade it against cost or time [25, 21, 24, 3].

Conclusion (C2). Greedy heuristics, ILP/SAT and NEMO preserve coverage under their standard configurations. Search-based and similarity-based methods require explicit configuration to guarantee C2.

5.3. Criterion C3: Reduction Effectiveness

C3 evaluates how strongly an algorithm reduces the test suite while satisfying C2.

ILP, SAT, and NEMO provide optimal or near-optimal reductions for the encoded objective [22, 7, 16], but at high computational cost. Among polynomial-time heuristics, EHGS, DG and related greedy variants consistently achieve strong reductions: EHGS and GRE typically reach around 72–74% suite-size reduction versus about 69% for HGS [10]; Jehan and Wotawa report roughly 70% reduction for multiple greedy variants on JavaScript projects with adequate mutation requirements [13]. DG often produces the smallest suites among greedy algorithms, at the cost of higher runtime [23]. Mutation-aware MOEAs also achieve substantial reductions, but their performance varies more strongly with configuration and subject programs [24].

MA-EHGS builds directly on EHGS's selection strategy, so its reduction behaviour follows the same pattern; adding mutation-awareness changes which tests are preferred in Phase 2 but does not alter the underlying SRM-based mechanism.

Conclusion (C3). Exact ILP/SAT/NEMO offer optimal reductions but are expensive. EHGS, DG and related greedy heuristics provide strong practical reduction; MA-EHGS inherits this behaviour from EHGS.

5.4. Criterion C4: Mutation Score Preservation

C4 measures how well an algorithm preserves mutation score as a proxy for fault-detection capability.

Coverage-only minimisation can lose mutants, especially under aggressive reduction. Noemmer and Haas report relative mutation-score losses between about 3.5% and 21%, depending on reduction thresholds and subject systems [18]. Jehan and Wotawa confirm that, under adequate reduction (100% coverage retained), loss is typically small but increases once coverage constraints are relaxed [13]. Many-objective evolutionary algorithms that include mutation score as an explicit objective retain 95–100% of mutants in their benchmarks [24].

These results suggest two key points: (i) structural redundancy does not imply semantic redundancy, and (ii) explicit mutation-awareness helps preserve fault-detection capability.

MA-EHGS addresses this by encoding mutants as explicit requirements and weighting them via parameter β, so that mutation information directly influences

test selection. In contrast, coverage-only greedy variants have no mechanism to prioritise tests that are mutation-critical but structurally similar to others.

Conclusion (C4). Mutation-aware greedy and evolutionary techniques provide the strongest mutation preservation. MA-EHGS offers a deterministic, SRM-based way to integrate mutation-awareness, whereas traditional coverage-only heuristics rely on structural coverage alone.

5.5. Criterion C5: Computational Scalability

C5 concerns how algorithms scale with increasing numbers of tests and requirements, as encountered in LASSO's SRMs.

Greedy SRM-based heuristics, including Classical Greedy, HGS, EHGS and MA-EHGS, operate in $O(nm)$ time where n is the number of tests and m the number of requirements [25, 18]. DG remains polynomial but incurs higher quadratic overhead due to concept-lattice construction [23]. These approaches are routinely reported as scalable in empirical studies [10, 18, 13].

ILP/SAT and NEMO, by contrast, face exponential worst-case complexity and are generally limited to smaller instances [7, 25, 16]. Search-based techniques and MOEAs require many evaluations of candidate suites across generations, leading to substantially higher runtime than greedy algorithms [25, 21, 24]. Similarity-based methods often rely on pairwise comparisons between tests, resulting in roughly $O(n^2m)$ behaviour, which can still be acceptable for prioritisation but is less attractive for frequent, large-scale minimisation [3].

Conclusion (C5). Greedy SRM-based heuristics (including MA-EHGS) provide the best scalability profile for large SRMs. Exact methods and metaheuristics are significantly more expensive.

5.6. Criterion C6: Implementation Feasibility

C6 assesses the practical effort and infrastructure required to integrate an algorithm into LASSO.

Greedy algorithms are deterministic, depend only on coverage and mutation matrices, and require no external solvers or complex runtime infrastructure [25, 18,

13]. EHGS and DG add only modest algorithmic complexity on top of basic greedy selection [23, 10]. MA-EHGS fits into the same pattern, extending EHGS with two weighting parameters (α and β) but leaving the overall data flow unchanged.

In contrast, ILP/SAT and NEMO require integration with external optimisation solvers [7, 25, 16]. Search-based techniques and MOEAs need an evolutionary framework, representations, and parameter tuning [25, 21, 24]. Similarity-based methods depend on clustering or similarity libraries and additional feature-engineering steps [3].

Conclusion (C6). SRM-based greedy heuristics, including MA-EHGS, are the easiest to integrate and maintain. Solver-based and evolutionary approaches impose substantially higher engineering overhead.

5.7. Selection Decision

Taken together, the criteria point to a clear trade-off landscape:

- **Greedy heuristics** (Classical Greedy, HGS, EHGS, DG) satisfy C1–C3 and C5–C6 well, but traditional variants are coverage-only with respect to C4.
- **Exact methods** (ILP, SAT, NEMO) excel in C2 and C3 but perform poorly in C5 and C6 due to computational and integration cost.
- **Search-based and MOEA approaches** support mutation-awareness (C4) and flexible objectives, but incur high runtime and tuning effort (C5–C6).
- **Similarity-based methods** are useful for prioritisation under time constraints, but less aligned with strict minimisation and deterministic reproducibility.

MA-EHGS combines the strengths of greedy SRM-based heuristics with explicit mutation-awareness:

- It operates directly on SRMs and requires no program structure (C1).
- It guarantees structural coverage (and, in this thesis, full mutant coverage) by treating all requirements as mandatory (C2).

- It inherits the strong reduction behaviour of EHGS (C3).
- It incorporates mutation information via β-weighted mutant requirements (C4).
- It retains greedy $O(nm)$ scalability (C5).
- It integrates into LASSO without external solvers or major architectural changes (C6).

Overall, MA-EHGS offers the most favourable balance across C1–C6 for LASSO's SRM-based environment: it preserves the simplicity and scalability of greedy minimisation while addressing the empirically observed coverage–mutation gap through explicit mutation-awareness. The next chapter presents its implementation and empirical evaluation.

6. Implementation

This chapter presents the technical implementation of the Mutation-Aware Enhanced HGS minimiser within LASSO. The work was developed in a dedicated branch:

<https://swtrepo.informatik.uni-mannheim.de/laith.sandouk/lasso/-/tree/test-suite-minimizatio

and the full reference implementation is available at:

m.de/laith.sandouk/lasso/-/blob/test-suite-minimization/lasso-llm/src/main/java/de/uni_mannheim/s

The contribution of this work lies in four areas: reliable per-test coverage collection, mutation-aware requirement extraction, a unified requirements-matrix representation, and a scalable realisation of the Enhanced HGS algorithm extended with mutation weighting. All components integrate seamlessly into the existing Arena–Mutation–SRM workflow and remain fully language-independent.

6.1. Architecture Overview

The implementation adds a cohesive data-processing layer that retrieves execution data from SRMs, converts them into requirement mappings, and applies the minimiser before writing back a reduced SRM. Central to this design is a unification layer that merges structural coverage and mutation information into a single requirements matrix. This preserves LASSO's language-agnostic design and enables consistent minimisation across different programming languages.

Three new components form the backbone of the implementation. The `SRMCoverageExtractor` collects per-test coverage from SRM sheets generated during Arena execution and standardises them using a language-neutral `COVERAGE_testName` prefix, replacing earlier tool-specific formats. The `MutationAnalyzer` reconstructs mutation kills from SRM outputs by comparing original and mutant variants cell-by-cell, resolving previous inaccuracies caused by single-cell comparisons or null handling. Finally, the `BitSetMatrix` provides a compact, high-performance representation

of all test–requirement relations, with structural elements and mutants encoded uniformly.

6.2. Data Extraction and Representation

Coverage is collected by executing JaCoCo instrumentation separately for each test, generating isolated coverage sheets per test. By adopting language-neutral identifiers, the design remains extensible to future Python or JavaScript coverage integrations.

Mutation information is reconstructed from SRM entries of type `value`. A mutant is considered killed if any output cell diverges from the original implementation:

$$\exists(x, y) : M_{i,\text{orig}}(x, y) \neq M_{i,m}(x, y).$$

This refined method yields a precise kill signal and corrects inconsistencies of earlier pipelines that observed only the first output cell.

	Original	**Mutant 1**	**Mutant 2**	**Mutant 3**
Test 1	42	41	42	43
Test 2	100	100	100	100
Test 3	true	false	true	false

Figure 6.1.: Example SRM for mutation score extraction. The first column shows outputs from the original implementation; subsequent columns show mutant variants. Green cells indicate outputs matching the original; red cells indicate divergent outputs (killed mutants). Test 1 kills Mutants 1 and 3. Test 2 kills no mutants. Test 3 kills Mutants 1 and 3. Tests are compared cell-by-cell across all output positions.

Coverage and mutation elements are unified into the set

$$R = R_{\text{coverage}} \cup R_{\text{mutation}},$$

which forms the input to the minimisation algorithm.

6.3. Algorithmic Realisation

The minimiser implements the two-phase Enhanced HGS algorithm. Phase 1 selects all tests that uniquely satisfy a requirement, ensuring that both structural and mutation-specific elements with cardinality one are always preserved. Phase 2 applies a weighted greedy selection:

$$\text{score}(t_i) = \alpha \cdot |\text{newCoverage}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|,$$

with default parameters $\alpha = 0.5$ and $\beta = 0.5$. BitSet operations keep runtime at $O(nm)$ and allow efficient experimentation with different scoring configurations.

6.4. Integration Into LASSO

The minimiser is exposed as the LSL action `TestSuiteMinimization`. It executes after both Arena and Mutation, consumes SRM data from Ignite, performs minimisation, and optionally replaces the SRM with a reduced version.

Listing 6.1 shows a representative configuration. Configurable parameters are shown in blue color.

Listing 6.1: LSL configuration for the minimisation action

```
action(name: 'minimize', type: 'TestSuiteMinimization') {
    dependsOn 'mutate'
    includeAbstractions 'Base64'

    alpha = 50
    beta = 50
    keepMutants = true
    keepOracle = false

}
```

The implementation required updating SRM extraction logic, correcting test-name normalisation, introducing language-agnostic sheet identifiers, and refining mutation comparison behaviour. These changes collectively make SRM-based minimisation correct, robust, and compatible with diverse software systems.

6.5. Validation

Validation confirmed that all structural and mutation requirements are preserved for $\alpha > 0$ and $\beta > 0$, and that the algorithm behaves deterministically under fixed configurations. Unit tests verify requirement mapping, kill-detection correctness, and algorithmic behaviour. Benchmarks (Appendix H.8) show strong reduction effectiveness (e.g., Base64: 80 tests $\rightarrow$ 8 tests) with full retention of structural and mutation requirements. As expected, MA-EHGS performs slightly less aggressively than coverage-only Enhanced HGS, but the difference is not critical.

7. Discussion

This chapter critically analyses test suite minimisation from two perspectives: general advantages and disadvantages inherent to the technique, followed by specific considerations for the Mutation-Aware Enhanced HGS implementation within LASSO's architectural constraints.

7.1. General Advantages and Disadvantages of Test Suite Minimisation

Test suite minimisation represents a sophisticated optimisation challenge where no single approach universally excels. This section examines inherent trade-offs and fundamental constraints independent of specific algorithmic implementations.

7.1.1. Practical Performance Benefits

Empirical evidence from large-scale industrial environments underscores the significant performance gains achievable through test suite minimisation. Bach et al. [3] observed that over 90% of test cases in a major industrial project (SAP) were redundant in terms of line coverage, with full test execution requiring several tens of hours. Applying minimisation techniques in such contexts can reduce execution time from many hours to just a few hours, or even minutes, without compromising coverage quality.

These reductions yield immediate and measurable benefits. A 50–75% shrinkage in test suites translates directly into lower computational demands, reduced infrastructure expenditure, and substantial cost savings. Minimised test execution drastically decreases CPU hours, memory allocation, and cloud consumption, which in turn cuts operational costs and energy usage. In continuous integration pipelines running hundreds of builds per day, the cumulative effect can be dra-

matic: a 60% reduction across 500 daily builds eliminates roughly 300 redundant test executions per cycle [25], saving both time and significant financial resources.

Shorter test cycles also enhance developer productivity. Compressing test execution times from hours to minutes accelerates feedback loops, allowing defects to surface earlier and be resolved faster. The ability to detect critical failures almost immediately reshapes development practices, especially in agile environments that rely on frequent integration and rapid iteration.

7.1.2. The Absence of a Perfect Solution

Despite these compelling advantages, test suite minimisation remains an inherently complex optimisation problem with no universally optimal solution. The difficulty arises from conflicting objectives and problem-specific constraints that vary across application domains. Minimisation confronts multiple competing goals: maximising reduction while preserving coverage, maintaining fault-detection capability, ensuring computational feasibility, and adapting to diverse system characteristics. No technique satisfies all objectives simultaneously across all contexts.

Different application domains prioritise these objectives differently. Safety-critical systems demand absolute fault-detection preservation, favouring conservative approaches that sacrifice reduction potential. High-frequency continuous integration pipelines prioritise execution speed, accepting greater risk of fault-detection loss. Large-scale observatories require language-agnostic solutions operating on behavioural abstractions, excluding techniques dependent on source code analysis.

This multi-objective optimisation creates a complex solution space where trade-offs vary by context. Conservative approaches produce larger minimised suites with stronger guarantees. Aggressive approaches achieve greater reduction with weaker assurances. The optimal balance depends on deployment context, testing objectives, and acceptable risk levels.

Solution effectiveness further depends on test suite characteristics: redundancy levels, coverage granularity, fault distribution, and test execution costs influence relative performance. Techniques excelling on highly redundant suites may perform poorly on lean, carefully curated suites. This context-dependence necessi-

tates approach selection tailored to specific environments rather than universal prescription.

7.1.3. The Coverage Limitation

Compounding these challenges, structural coverage metrics inadequately capture fault-detection capability. Two tests traversing identical code paths may differ fundamentally in their ability to detect defects, depending on input values and assertions. Coverage preservation therefore guarantees structural but not behavioural equivalence. Tests appearing coverage-redundant may be semantically distinct in fault-revealing behaviour.

Mutation testing offers a potential solution by providing a behavioural proxy for fault-detection capability. Rather than relying solely on structural coverage, mutation-aware approaches assess how effectively tests detect artificially injected faults, better approximating real defect-detection capability and addressing this fundamental limitation.

7.2. Thesis-Specific Considerations for MA-EHGS in LASSO

Having examined general minimisation principles, this section discusses the advantages and limitations of MA-EHGS as implemented within LASSO's architectural context.

7.2.1. Advantages of the Approach

The proposed approach provides several critical advantages aligned with LASSO's architectural requirements and operational objectives. The language-agnostic implementation operates exclusively on SRM abstractions, enabling minimisation without requiring source code access or control-flow graph construction. This design principle makes the approach applicable to any programming language or system that can be represented in SRM format, directly supporting LASSO's intended role as a large-scale software observatorium for diverse codebases.

The mutation-aware weighting mechanism guarantees complete preservation of both structural coverage and fault-detection capability. As long as neither α (coverage weight) nor β (mutation weight) equals zero, the algorithm ensures 100% coverage retention and 100% mutation score retention [10]. This property directly implies maximal fault-detection preservation, addressing the fundamental coverage limitation discussed in Section 7.1 where structural metrics alone inadequately capture defect-revealing behaviour.

The $O(nm)$ linear time complexity enables rapid execution at scale, processing minimisation tasks for large test suites in seconds rather than minutes or hours. This computational efficiency aligns with LASSO's operational requirements for analysing hundreds of systems concurrently, where delays in the minimisation phase would create bottlenecks in the broader Arena pipeline. Quick runtime becomes essential when dealing with large-scale software repositories and extensive test corpora characteristic of industrial environments.

7.2.2. Limitations of the Implementation

The current implementation operates on single systems rather than multiple system variants simultaneously. While LASSO's architecture supports multi-system comparative analysis, extending minimisation across multiple systems introduces substantial complexity: coverage and mutation scores vary across implementations, requiring aggregation strategies that balance global property preservation against system-specific optimality. Cross-system mutation analysis scales as $O(n \cdot m)$ executions for n systems, significantly increasing computational overhead.

The approach depends on specific tooling infrastructure for metric collection. Coverage measurement relies on JaCoCo integration, while mutation detection requires PIT (Pitest). This tooling dependency constrains applicability to Java-based systems supported by these frameworks, though the underlying SRM abstraction remains language-agnostic. Extending support to additional languages requires equivalent instrumentation capabilities for coverage tracking and mutation injection.

Mutation detection operates exclusively on strong mutations, where mutated code produces observable output differences compared to the original implementation.

Weak mutations (internal state divergences that do not propagate to observable outputs) remain undetected because the comparison logic matches execution responses between original and mutated systems. Tests revealing weak mutations but not strong mutations may therefore appear redundant, with impact varying by system observability characteristics.

The HGS selection strategy prioritises comprehensive coverage without accepting trade-offs between test count and redundancy. When test t_1 covers requirements $\{1, \ldots, 20\}$ and test t_2 covers requirements $\{1, \ldots, 21\}$, both tests are retained because t_2 provides unique coverage of requirement 21. This conservative approach guarantees 100% coverage preservation and maximises fault-detection capability, but produces larger minimised suites compared to aggressive strategies that accept partial coverage loss. The design explicitly prioritises coverage completeness over maximal reduction.

The current approach and implementation were validated similarly to unit tests in object-oriented programming, lacking comprehensive integration testing. While unit-test validation ensures algorithmic correctness and basic functionality, it does not assess end-to-end performance in real LASSO pipelines, potentially overlooking issues in data extraction, SRM generation, or multi-system interactions.

7.2.3. Implications for LASSO Platform Evolution

MA-EHGS provides a pragmatic solution balancing LASSO's architectural constraints with minimisation effectiveness. The mutation-aware design achieves substantial efficiency gains while preserving fault-detection capability. As expected, incorporating mutation constraints yields slightly lower reduction than coverage-only Enhanced HGS, but the trade-off is not critical given the improved fault-detection guarantees. The extension of the empirically validated Enhanced HGS baseline [10] demonstrates that language-agnostic behavioural abstraction enables practical large-scale optimisation.

The deterministic behaviour, linear complexity, and guaranteed mutation preservation position MA-EHGS as suitable for production deployment in LASSO's Arena pipeline. While constrained by strong mutation limitations and parameter sensitivity, the configurable design accommodates diverse user preferences, from pure coverage-based minimisation ($\beta = 0$) to pure mutation-based minimi-

sation ($\alpha = 0$), acknowledging that optimal configurations reflect domain-specific trade-offs rather than universal prescriptions.

7.3. Future Work

The limitations identified in Section 7.2 highlight several directions for extending the MA-EHGS approach and improving its applicability within LASSO.

7.3.1. Multi-Language Support

The current prototype relies on Java tooling (JaCoCo and PIT). Extending MA-EHGS to Python, JavaScript, and other ecosystems requires integrating equivalent coverage and mutation frameworks and normalising their outputs into a unified SRM representation. Key challenges include ensuring metric granularity consistency and mapping language-specific mutation operators.

7.3.2. Multisystem Minimisation

LASSO's multi-system SRMs motivate minimisation across several implementations simultaneously. Future work should investigate strategies for aggregating coverage and mutation information across systems, balancing global preservation with computational feasibility. This includes studying conservative union-based vs. intersection-based preservation strategies and quantifying the increased overhead of cross-system mutation execution.

7.3.3. Integration Testing Validation

Validation has so far focused on isolated unit tests. End-to-end integration tests within LASSO's Arena pipeline are needed to evaluate data extraction reliability, SRM generation correctness, and performance under realistic workloads (large-scale SRMs and many parallel systems). This ensures robustness for production deployment.

7.3.4. Alpha–Beta Benchmarking

Empirical parameter sensitivity analysis on the Base64 benchmark revealed minimal impact of α–β configurations on minimisation outcomes. Testing multiple parameter combinations produced identical 8-test results for all non-edge cases, with only $\alpha = 0$ or $\beta = 0$ configurations yielding slightly larger suites (7 tests). This parameter insensitivity simplifies configuration, as the balanced defaults $\alpha = 0.5$ and $\beta = 0.5$ perform equivalently to other weighted combinations for this benchmark.

However, this finding is based on relatively small test suites (80 tests for Base64). Larger test suites with more complex coverage patterns may exhibit greater parameter sensitivity, particularly when $\alpha + \beta > 1$ amplifies both coverage and mutation contributions simultaneously. A systematic grid search across parameter combinations on diverse system types and scales could identify configurations that optimise reduction while preserving coverage and mutation scores. Such benchmarking could be integrated as an LSL action to support empirically guided parameter selection.

7.3.5. Weak Mutation Support

MA-EHGS currently considers only strong mutants. Capturing weak mutants would require deeper instrumentation to detect internal state divergences. Although more costly, this would provide finer behavioural insight and improve minimisation quality for systems with complex internal logic.

7.3.6. LLM-Assisted Gap Analysis

Large language models could assist in identifying and generating tests for uncovered coverage or mutation gaps. By analysing SRM data and optional code context, LLMs could propose targeted tests which are subsequently validated automatically. This complements minimisation by enabling systematic test suite augmentation.

8. Conclusion

8.1. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO by selecting and implementing the Mutation-Aware Enhanced Harrold–Gupta–Soffa (MA-EHGS) approach. Based on a systematic literature review and evaluation criteria C1–C6, MA-EHGS was identified as the most suitable technique due to its scalability, preservation guarantees, and compatibility with LASSO's behavioural abstraction. Integrated into the analysis pipeline, MA-EHGS combines line, branch, and method coverages with mutation information. In our benchmarking (mainly multiple Base64 implementations), MA-EHGS achieved strong reductions while preserving 100% coverage for all $\alpha > 0$ and 100% mutation retention for all $\beta > 0$ configurations. As expected, reduction effectiveness is slightly lower than coverage-only Enhanced HGS due to additional mutation constraints, but the difference is not critical. This behaviour follows directly from the HGS selection mechanism. Unit-test validation confirmed correct algorithmic behaviour, with full pipeline evaluation remaining for future work.

8.2. Key Contributions

The thesis makes three main contributions:

Algorithmic contribution. MA-EHGS extends the validated Enhanced Harrold–Gupta–Soffa (EHGS) algorithm [10] with a dual-parameter weighting scheme that jointly considers structural coverage and mutation preservation:

$$\text{score}(t) = \alpha \cdot |\text{newCoverage}| + \beta \cdot |\text{newMutants}|$$

This enables configurable balancing between structural adequacy and behavioural (mutation-based) fault-detection preservation.

Practical integration into LASSO. The framework extracts metric data from SRM response objects, performs minimisation, and generates reduced SRMs while preserving schema and language independence.

8.3. Future Work

Future improvements (detailed in Section 7.3) include extending the approach beyond Java, enabling multisystem minimisation for cross-project SRMs, conducting full integration testing in LASSO's Arena pipeline, benchmarking α–β configurations, supporting weak mutation detection, and exploring LLM-assisted test generation to close coverage or mutation gaps.

Research Questions Answered

RQ1: Greedy heuristics, especially EHGS, were identified as the most effective and practical techniques in current literature.

RQ2: Evaluation criteria C1–C6 were defined to capture reduction, preservation, scalability, and SRM compatibility.

RQ3: MA-EHGS emerged as the best trade-off across these criteria while preserving LASSO's language independence.

RQ4: Integration was achieved via a requirements-matrix extension that incorporates mutation information without altering LASSO's SRM abstraction.

Overall, this thesis demonstrates that SRM-based minimisation can substantially reduce test suites while preserving fault-detection capability. The mutation-aware MA-EHGS algorithm therefore provides a scalable and configurable mechanism for improving testing efficiency in large software repositories and strengthens the case for LASSO's behavioural abstraction as a foundation for future large-scale software analysis.

Bibliography

[1] Neeru Ahuja and Pradeep Kumar Bhatia. «Test Suite Minimization Through PSO». In: *Proceedings of the International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)*. IEEE. 2024.

[2] Miltiadis Allamanis et al. «A Survey of Machine Learning for Big Code and Naturalness». In: *ACM Comput. Surv.* 51.4 (July 2018). ISSN: 0360-0300. DOI: 10.1145/3212695. URL: https://doi.org/10.1145/3212695.

[3] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. «Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project». In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 2017, pp. 3–12. DOI: 10.1109/ICSTW.2017.6.

[4] Barry W. Boehm. «Software Engineering Economics». In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.

[5] T.Y. Chen and M.F. Lau. «A new heuristic for test suite reduction». In: *Information and Software Technology* 40.5 (1998), pp. 347–354. ISSN: 0950-5849. DOI: https://doi.org/10.1016/S0950-5849(98)00050-0. URL: https://www.sciencedirect.com/science/article/pii/S0950584998000500.

[6] V. Chvatal. «A Greedy Heuristic for the Set-Covering Problem». In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235. ISSN: 0364765X, 15265471. URL: http://www.jstor.org/stable/3689577 (visited on Nov. 10, 2025).

[7] S. Elbaum, A.G. Malishevsky, and G. Rothermel. «Test case prioritization: a family of empirical studies». In: *IEEE Transactions on Software Engineering* 28.2 (2002), pp. 159–182. DOI: 10.1109/32.988497.

[8] Gordon Fraser and Andrea Arcuri. «EvoSuite: automatic test suite generation for object-oriented software». In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ESEC/FSE '11. Szeged, Hungary: Association for Computing Machinery, 2011, pp. 416–419. ISBN: 9781450304436. DOI: 10.1145/2025113.2025179. URL: https://doi.org/10.1145/2025113.2025179.

[9] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[10] Angelin Gladston et al. «Test suite reduction using HGS based heuristic approach». In: *Computing and Informatics* 34 (Jan. 2015), pp. 1113–1132.

[11] M.J. Harrold, R. Gupta, and M.L. Soffa. «A methodology for controlling the size of a test suite». In: *Proceedings. Conference on Software Maintenance 1990*. 1990, pp. 302–310. DOI: 10.1109/ICSM.1990.131378.

[12] Hadi Hemmati, Andrea Arcuri, and Lionel Briand. «Achieving scalable model-based testing through test case diversity». In: *ACM Trans. Softw. Eng. Methodol.* 22.1 (Mar. 2013). ISSN: 1049-331X. DOI: 10.1145/2430536.2430540. URL: https://doi.org/10.1145/2430536.2430540.

[13] Seema Jehan and Franz Wotawa. «An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage». In: *IEEE Access* 11 (2023), pp. 65427–65442. DOI: 10.1109/ACCESS.2023.3289073.

[14] Yue Jia and Mark Harman. «An Analysis and Survey of the Development of Mutation Testing». In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678. DOI: 10.1109/TSE.2010.62.

[15] Marcus Kessel. «LASSO - an observatorium for the dynamic selection, analysis and comparison of software». PhD thesis. Feb. 2023.

[16] Jun-Wei Lin et al. «NEMO: Multi-Criteria Test-Suite Minimization with Integer Nonlinear Programming». In: *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM. 2018, pp. 658–668.

[17] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. «REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization». In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[18] Raphael Noemmer and Roman Haas. «An Evaluation of Test Suite Minimization Techniques». In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.

[19] A. Jefferson Offutt and Ronald H. Untch. «Mutation 2000: uniting the orthogonal». In: *Mutation Testing for the New Century*. USA: Kluwer Academic Publishers, 2001, pp. 34–44. ISBN: 0792373235.

[20] Carlos Pacheco et al. «Feedback-Directed Random Test Generation». In: *29th International Conference on Software Engineering (ICSE'07)*. 2007, pp. 75–84. DOI: 10.1109/ICSE.2007.37.

[21] Saif Ur Rehman Khan et al. «A Systematic Review on Test Suite Reduction: Approaches, Experiment's Quality Evaluation, and Guidelines». In: *IEEE Access* 6 (2018), pp. 11816–11841. DOI: 10.1109/ACCESS.2018.2809600.

[22] G. Rothermel and M.J. Harrold. «Empirical studies of a safe regression test selection technique». In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419. DOI: 10.1109/32.689399.

[23] Sriraman Tallam and Neelam Gupta. «A concept analysis inspired greedy algorithm for test suite minimization». In: vol. 31. Sept. 2005, pp. 35–42. DOI: 10.1145/1108792.1108802.

[24] Zheng Wei et al. «Test Suite Minimization with Mutation Testing-Based Many-Objective Evolutionary Optimization». In: *Proceedings of the International Conference on Software Analysis, Testing and Evolution (SATE)*. IEEE. 2017.

[25] S. Yoo and M. Harman. «Regression testing minimization, selection and prioritization: a survey». In: *Softw. Test. Verif. Reliab.* 22.2 (Mar. 2012), pp. 67–120. ISSN: 0960-0833. DOI: 10.1002/stv.430. URL: https://doi.org/10.1002/stv.430.

[26] Hong Zhu, Patrick A. V. Hall, and John H. R. May. «Software unit test coverage and adequacy». In: *ACM Comput. Surv.* 29.4 (Dec. 1997), pp. 366–427. ISSN: 0360-0300. DOI: 10.1145/267580.267590. URL: https://doi.org/10.1145/267580.267590.

Appendix

Detailed Evaluation Tables for Comparative Analysis

This appendix provides the detailed evaluation tables and justifications for the algorithm scores discussed in Chapter 5. The tables are aligned with the technique families and criteria introduced in Chapters 3 and 5 and focus on approaches that can be expressed on a Stimulus–Response Matrix (SRM).

The following algorithms and families are considered:

- Classical Greedy (CG)
- Harrold–Gupta–Soffa (HGS)
- Delayed Greedy (DG)
- Enhanced HGS (EHGS)
- Mutation-Aware Enhanced HGS (MA-EHGS; this thesis)
- Exact ILP/SAT set-cover formulations (ILP)
- NEMO (nonlinear integer programming) [16]
- Mutation-aware many-objective evolutionary algorithms (MOEA-mu; Wei et al. [24])
- Coverage-based search-based techniques (GA/PSO/ABC)
- Similarity- and overlap-based methods (SIM/OV; e.g. Bach et al. [3], Khan et al. [21])
- REDUNET (set-cover + CFG-based optimisation) [17]

Scores for C2–C6 use a scale from 0 (poor) to 5 (very strong). C1 is a mandatory SRM-compatibility check (pass/fail).

A.1. C1: SRM Compatibility

Table 1.: C1: SRM Compatibility (Stimulus–Response Matrix only).

Algorithm / Family	Pass	Justification and Source
Classical Greedy (CG)	✓	Standard greedy set-cover heuristics operate on a test–requirement incidence matrix only [6, 25, 18].
HGS	✓	HGS uses requirement cardinalities and coverage sets. No CFG or AST information is required [11, 25].
Delayed Greedy (DG)	✓	Uses concept analysis over coverage relationships among tests. Inputs are coverage sets or a matrix [23].
Enhanced HGS (EHGS)	✓	Evaluated on coverage matrices (FIRST RUN benchmark) and defined purely in terms of test–requirement incidence [10].
MA-EHGS	✓	This thesis: extends EHGS by treating mutants as additional requirement columns in the same matrix.
Exact ILP/SAT (ILP)	✓	ILP/SAT encodings take coverage matrices as input and formulate minimal hitting-set constraints [7, 25].
NEMO	✓	NEMO adds nonlinear multi-criteria objectives (e.g. cost, fault detection) but still uses coverage information as input [16].
MOEA-mu	✓	Many-objective EAs use coverage and mutation vectors to define multi-objective fitness functions [24].
Coverage-based GA/PSO/ABC	✓	Evolutionary/swarm-based minimisation typically encodes candidates as subsets of tests and evaluates them via coverage vectors [25, 21].
SIM/OV (similarity & overlap)	✓	Similarity- and overlap-based methods compute distances or clusters over coverage vectors or bitsets [3, 21].
REDUNET	×	REDUNET explicitly relies on control-flow graphs and program-structure information to reason about redundancy [17]. It is therefore not applicable in SRM-only settings.

C1 Justification

C1 enforces that algorithms must work purely on SRM-style test–requirement information without access to source code or CFGs. This is necessary for LASSO's language-agnostic, black-box architecture. All matrix-based formulations (greedy, exact, search-based, similarity-based) pass. REDUNET fails due to its CFG dependency and is excluded from further comparison.

B.2. C2: Coverage Preservation

Table 2.: C2: Structural Coverage Preservation (0–5).

Algorithm / Family	Score	Evidence and Rationale
Classical Greedy (CG)	5/5	Standard greedy set-cover algorithms iterate until all requirements are covered [6, 25]. In regression-test minimisation, they are used with adequacy constraints (e.g. all statements covered) [22, 25, 18].
HGS	5/5	HGS selects tests until every requirement has at least one covering test [11]. Empirical studies treat it as coverage-preserving under adequate settings [25, 18].
Delayed Greedy (DG)	5/5	Tallam & Gupta remove subsumed tests, then apply greedy selection. Coverage is preserved by construction [23]. Jehan & Wotawa use DG under full mutant requirements [13].
Enhanced HGS (EHGS)	5/5	EHGS first selects tests with unique coverage, then greedily covers remaining requirements [10]. Reduced suites satisfy the same coverage criteria as originals.
MA-EHGS	5/5	In this thesis, structural coverage and (selected) mutants are treated as requirements. MA-EHGS continues selecting tests until all requirements in this set are covered. As long as all desired coverage elements and mutants are included in the requirement set, they are preserved by construction in the minimised suite.
Exact ILP/SAT (ILP)	5/5	ILP/SAT formulations enforce coverage as hard constraints (each requirement must be covered at least once) [7, 25]. Any feasible solution preserves adequacy.
NEMO	5/5	NEMO extends ILP with nonlinear objectives but still imposes coverage constraints, producing suites that meet predefined coverage thresholds [16].
MOEA-mu	4/5	Wei et al. treat statement coverage as one objective among several (including mutation score and cost) [24]. Their best solutions maintain statement coverage, but coverage is not a hard constraint in the algorithm design.
Coverage-based GA/PSO/ABC	3/5	Many GA/PSO/ABC approaches maximise coverage together with size or cost but do not guarantee 100% coverage [25, 21]. Coverage may be traded off against other objectives.
SIM/OV	2/5	Similarity- and overlap-based methods (e.g. Bach et al.) aim to maximise coverage *within time budgets*, not to enforce full adequacy [3]. Coverage depends on budget and thresholds.

C2 Discussion

Greedy and exact matrix-based algorithms allow LASSO to guarantee structural adequacy (100% of the chosen coverage metric) in SRM form. MA-EHGS preserves this guarantee for all requirements included in the matrix while extending the requirement set to include mutants.

C.3. C3: Reduction Effectiveness

Table 3.: C3: Reduction Effectiveness (0–5).

Algorithm / Family	Score	Empirical Behaviour and Evidence
Exact ILP/SAT (ILP)	5/5	ILP/SAT formulations compute minimal hitting sets for the encoded constraints [7, 25]. For a given objective (e.g. minimising number of tests), they provide optimal reductions.
NEMO	5/5	NEMO optimises nonlinear multi-criteria objectives (cost, coverage, additional metrics) and can outperform linear ILP baselines in fault detection at comparable suite size [16]. For reduction relative to constraints, it is again optimal for its model.
Classical Greedy (CG)	4/5	Greedy set-cover heuristics provide a logarithmic approximation bound [6]. Noemmer & Haas and Jehan & Wotawa report substantial reductions (often around two-thirds to three-quarters of tests removed under adequate settings) in Java and JavaScript projects [18, 13].
HGS	4/5	HGS yields reductions comparable to classical greedy in empirical studies [25, 18]. Jehan & Wotawa find only minor differences in average reduction between greedy variants when all mutants must be killed [13].
Delayed Greedy (DG)	4/5	Tallam & Gupta show that DG often produces suites that are at least as small as those from classical greedy/HGS by removing subsumed tests first [23]. Jehan & Wotawa confirm that DG typically gives the smallest or near-smallest suites among their greedy variants [13].
Enhanced HGS (EHGS)	4/5	On the FIRST RUN benchmark, EHGS achieves an average suite-size reduction of 73.66% compared to 69.01% for HGS and 73.51% for GRE [10]. This suggests strong reduction capability, at least on the studied program.
MA-EHGS	4/5	MA-EHGS inherits EHGS's two-phase structure and thus similar reduction potential, but may trade a small amount of reduction for better mutation preservation when β is emphasised. Empirical results in this thesis show strong reduction effectiveness on instructional systems while preserving all structural and mutation requirements (see Section H.8). As expected, MA-EHGS performs slightly less aggressively than coverage-only Enhanced HGS, but the difference is not critical.
MOEA-mu	4/5	Wei et al. show that many-objective EAs can significantly reduce test cost while maintaining coverage and fault detection [24]. The exact reduction depends on subject programs and chosen solutions on the Pareto front.
Coverage-based GA/PSO/ABC	3/5	Coverage-driven GA/PSO/ABC approaches achieve notable reductions but are more sensitive to parameter tuning and may fall short of greedy or ILP baselines in some studies [25, 21].
SIM/OV	2/5	Similarity- and overlap-based methods in industrial settings (e.g. at SAP) primarily aim to improve coverage within a fixed time budget rather than to compute minimal suites [3]. Reductions are moderate and tuned to execution-time constraints.

C3 Discussion

Exact ILP/SAT and NEMO provide optimal or near-optimal reductions relative to their models but at higher computational cost. Greedy heuristics (CG, HGS, DG, EHGS, MA-EHGS) consistently provide strong practical reductions (often around 70% in empirical studies) while remaining efficient. Metaheuristics and similarity-based methods show variable but often useful reductions.

D.4. C4: Mutation Score Preservation

Table 4.: C4: Mutation Score Preservation (0–5).

Algorithm / Family	Score	Behaviour and Evidence
MA-EHGS	5/5	MA-EHGS treats mutants as explicit requirements and weights them via β. In LASSO, the algorithm can be configured so that all mutants of interest (e.g. all mutants killed by the original suite that are not equivalent or flaky) are included in the requirement set, in which case the minimised suite continues to kill all of them. In practice, experiments in this thesis demonstrate complete mutation retention (100%) while still achieving substantial reduction, which is stronger mutation support than coverage-only greedy baselines.
MOEA-mu	5/5	Wei et al. explicitly include mutation score as an optimisation objective. Their best solutions significantly reduce cost without reducing fault-detection ability relative to the original suite [24].
Classical Greedy (CG)	3/5	Coverage-only greedy minimisation can reduce mutation score when coverage thresholds are relaxed: Noemmer & Haas report relative mutation score losses between 3.5% and 21% depending on project and reduction level [18]. When all mutants are treated as requirements, Jehan & Wotawa show that greedy minimisation can achieve substantial reduction without compromising mutation-based fault detection [13].
HGS	3/5	As another coverage-only greedy heuristic, HGS shares CG's strengths and weaknesses. Noemmer & Haas report mutation score losses under coverage-based thresholds, while Jehan & Wotawa show no observed loss when all mutants must be killed [18, 13].
Delayed Greedy (DG)	3/5	DG itself is coverage-based. Jehan & Wotawa find that, with all mutation requirements enforced, DG reduces suite size on average (around 70%) without lowering the number of killed mutants compared to the full suite [13]. Under relaxed coverage-only settings, the same coverage–mutation gap as for CG/HGS is expected.
Enhanced HGS (EHGS)	1/5	EHGS focuses on structural coverage only. The original evaluation reports coverage-based results but does not investigate mutation score [10]. Mutation preservation is therefore unknown.
Exact ILP/SAT (ILP)	1/5	ILP/SAT formulations in the literature minimise suite size subject to structural coverage constraints [7, 25]. Mutation score is not modelled and empirical mutation analyses are not reported.
NEMO	1/5	NEMO adds objectives for cost and fault detection (measured via historical failure data and coverage) but does not explicitly use mutation testing [16]. Mutation score effects are not evaluated.
Coverage-based GA/PSO/ABC	1/5	GA/PSO/ABC approaches cited in surveys focus on coverage and cost objectives [25, 21]. To the best of our knowledge, no empirical mutation-testing evaluations are reported in these works.
SIM/OV	1/5	Similarity- and overlap-based methods detect redun-

C4 Discussion: The Coverage–Mutation Gap

The literature documents a clear gap between structural coverage and mutation score:

- Noemmer & Haas show that coverage-based minimisation can lead to relative mutation score losses between 3.5% and 21% depending on subject and reduction level, even when structural adequacy is preserved [18].
- Jehan & Wotawa demonstrate that, when all mutants are treated as requirements, several greedy algorithms reduce suites by around 70% *without* compromising mutation-based fault detection [13].
- Wei et al. show that explicitly including mutation score as an objective allows many-objective EAs to reduce cost while maintaining defect detection ability [24].

MA-EHGS is designed to incorporate mutation information in a deterministic, SRM-based greedy algorithm and can be configured to preserve mutants of interest while still achieving meaningful reduction.

E.5. C5: Computational Scalability

Table 5.: C5: Computational Scalability (0–5).

Algorithm / Family	Score	Complexity and Practical Considerations
Classical Greedy (CG)	5/5	Greedy set-cover heuristics have polynomial complexity and are widely used on real-world projects [25, 18]. In practice, they scale to test suites with thousands of tests and requirements.
HGS	4/5	Harrold et al. give a worst-case bound of $O(n(n + m)\,MAXCARD)$ for n associated testing sets, m test cases, and maximal requirement cardinality MAXCARD [11]. This is still polynomial but somewhat heavier than simple greedy.
Delayed Greedy (DG)	3/5	DG includes concept lattice construction over coverage relationships [23]. Tallam & Gupta emphasise polynomial complexity, but the additional structure construction incurs overhead. Jehan & Wotawa observe higher runtimes than simpler greedy variants [13].
Enhanced HGS (EHGS)	4/5	EHGS performs a unique-coverage phase and then a greedy phase over the coverage matrix [10]. Its operations are polynomial in the number of tests and requirements and similar in nature to CG/HGS.
MA-EHGS	4/5	MA-EHGS adds mutation requirements and a weighted scoring function to EHGS but retains the same overall structure. A straightforward implementation is polynomial in the number of tests and requirements and behaves like other matrix-based greedy heuristics in practice.
Exact ILP/SAT (ILP)	1/5	Set-cover is NP-complete [9]. ILP/SAT encodings therefore have exponential worst-case complexity. They can handle small- to medium-sized instances [7, 25] but do not scale well to very large SRMs.
NEMO	1/5	NEMO's nonlinear formulation increases the number of variables and constraints substantially. Lin et al. report much higher optimisation times than for linear ILP [16]. Scalability is more limited than for greedy heuristics.
MOEA-mu	2/5	Many-objective EAs evaluate populations of candidate suites over many generations [24]. This leads to significantly higher computational cost than a single greedy pass, although experiments remain feasible for studied subjects.
Coverage-based GA/PSO/ABC	2/5	Similar to MOEA-mu, coverage-based search methods require many fitness evaluations and are more expensive than greedy heuristics [25, 21].
SIM/OV	4/5	Similarity- and overlap-based methods compute distances or clusters over coverage vectors. This is polynomial and typically dominated by test-execution time. Bach et al. report acceptable overhead for very large industrial suites [3].

F.6. C6: Implementation Feasibility

Table 6.: C6: Implementation Feasibility (0–5).

Algorithm / Family	Score	Practical Integration Aspects
Classical Greedy (CG)	5/5	Conceptually simple, deterministic, no parameter tuning. Requires only iteration over a coverage matrix.
HGS	4/5	Requires cardinality-based grouping and ordering of tests and requirements [11], but still straightforward to implement in a matrix-based setting.
Delayed Greedy (DG)	3/5	Concept lattice construction and manipulation are more complex and often rely on specialised data structures or libraries [23].
Enhanced HGS (EHGS)	4/5	Adds a unique-coverage phase and non-redundancy checks to HGS [10], but remains deterministic and matrix-based with modest implementation effort.
MA-EHGS	4/5	Extends EHGS with two scalar parameters (α, β) and a weighted scoring function. It is deterministic and integrates directly with LASSO's SRMs without external dependencies. A small grid search over β is sufficient for tuning.
Exact ILP/SAT (ILP)	1/5	Requires modelling the problem in ILP/SAT form and integrating an external solver (e.g. CPLEX, Gurobi, SAT solvers), including configuration and error handling [7, 25].
NEMO	1/5	Adds nonlinear constraints and auxiliary variables on top of ILP, requiring more complex solver configuration and model management [16].
MOEA-mu	2/5	Needs a multi-objective EA framework (e.g. NSGA-II, MOEA/D) and multiple parameters (population size, generations, operators), plus handling of non-determinism [24].
Coverage-based GA/PSO/ABC	2/5	Similar to MOEA-mu, but typically with fewer objectives. Still requires evolutionary/swarm framework and parameter tuning [25, 21].
SIM/OV	3/5	Requires coverage extraction plus similarity or clustering libraries [3, 21]. Integration effort is moderate but higher than for simple greedy heuristics.

G.7. Complete Multi-Criteria Scoring Table

Table 7 summarises the scores for C2–C6. C1 is treated as a mandatory SRM-compatibility check. All algorithms except REDUNET pass C1.

Table 7.: Multi-criteria scores for all algorithms (C2–C6).

Algorithm / Family	C2	C3	C4	C5	C6	Total
Classical Greedy (CG)	5	4	3	5	5	22
HGS	5	4	3	4	4	20
Delayed Greedy (DG)	5	4	3	3	3	18
Enhanced HGS (EHGS)	5	4	1	4	4	18
MA-EHGS	5	4	5	4	4	22
Exact ILP/SAT (ILP)	5	5	1	1	1	13
NEMO	5	5	1	1	1	13
MOEA-mu	4	4	5	2	2	17
GA/PSO/ABC	3	3	1	2	2	11
SIM/OV	2	2	1	4	3	12
REDUNET	Excluded: fails C1 (requires CFG)					

H.8. MA-EHGS Selection Justification

The multi-criteria analysis shows that several algorithms perform well:

- **Greedy heuristics** (CG, HGS, DG, EHGS) offer excellent coverage preservation (C2), strong reductions (C3), good scalability (C5), and high implementation feasibility (C6) [11, 10, 25, 18, 13].
- **Exact methods** (ILP, NEMO) are optimal for their formulations (C3) but limited by scalability and integration complexity (C5–C6) [7, 25, 16].
- **MOEA-mu** excels in mutation-awareness (C4) while offering good reduction (C3), but at higher computational and implementation cost [24].
- **Similarity-/overlap-based methods** (SIM/OV) scale well (C5) and integrate reasonably (C6), but target prioritisation under time budgets rather than strict minimisation [3, 21].

Classical Greedy and MA-EHGS both reach a total of 22/25. The decisive difference lies in C4 (mutation score preservation):

- **Classical Greedy:** Mutation is not modelled explicitly. Empirical work shows that coverage-only minimisation can lead to noticeable mutation score losses when coverage thresholds or requirements are relaxed [18]. When all mutants are treated as requirements, greedy algorithms can avoid mutation losses [13], but this is a property of the configuration rather than of the algorithm design.

- **MA-EHGS:** Treats mutants as explicit requirements and supports weighting them via β. In LASSO's configuration, MA-EHGS is used to incorporate mutation information directly into selection decisions and can be configured to retain the mutants of interest while still yielding strong reduction, as demonstrated on the case studies (Section H.8).

For LASSO's SRM setting, this thesis therefore selects **MA-EHGS** as the primary minimisation algorithm, justified by:

- SRM-only operation (C1),
- full structural coverage preservation for the chosen coverage metric (C2),
- strong reduction effectiveness comparable to other greedy heuristics (C3),
- explicit support for mutation information and very high mutation retention (C4),
- polynomial scalability suitable for large SRMs (C5),
- and straightforward integration with LASSO's existing architecture (C6).

Classical Greedy remains a natural baseline and backup option, especially when mutation information is unavailable.

Mutation-Aware Enhanced HGS Pseudocode

This appendix provides precise pseudocode for the Mutation-Aware Enhanced HGS (MA-EHGS) algorithm. The notation follows requirements-matrix formulations as used for EHGS [10] and other set-cover-based techniques.

Notation

- $T = \{t_1, \ldots, t_n\}$: set of tests (rows).
- $R_{\text{struct}} = \{r_1, \ldots, r_m\}$: structural requirements (e.g. lines, branches, methods).
- $R_{\text{mut}} = \{m_1, \ldots, m_p\}$: mutation requirements (mutants killed by the original suite).
- $R = R_{\text{struct}} \cup R_{\text{mut}}$: unified requirement set (columns).
- $M \in \{0,1\}^{n \times |R|}$: binary requirements matrix; $M_{ij} = 1$ iff test t_i satisfies requirement r_j.
- $R_i = \{r_j \in R \mid M_{ij} = 1\}$: set of requirements covered by test t_i.
- Weights: $\alpha \in [0,1]$ for structural requirements, $\beta \geq 0$ for mutants (default: $\alpha = 0.5$, $\beta = 0.5$). Setting $\beta = 0$ recovers a purely coverage-based EHGS. Setting $\alpha = 0$ would prioritise mutants only.

Algorithm

Procedure MAEHGS(M, α, β)
Input: Requirements matrix $M \in \{0,1\}^{n \times |R|}$, weights $\alpha \in [0,1]$, $\beta \geq 0$
Output: Selected test subset $S \subseteq T$

Phase 1: Essential Test Selection

1. For each requirement $r_j \in R$, compute its set of covering tests:

$$T_j = \{t_i \in T \mid M_{ij} = 1\}, \quad |T_j| = \text{cardinality of } T_j$$

2. Initialise selected tests and uncovered requirements:

$$S \leftarrow \emptyset, \quad R_{\text{uncov}} \leftarrow R$$

3. For each requirement $r_j \in R$ with $|T_j| = 1$:

   - Let t_i be the unique test in T_j.
   - Add t_i to S (if not already selected).
   - Remove all requirements covered by t_i:

$$R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_i$$

Phase 2: Weighted Greedy Selection

4. While $R_{\text{uncov}} \neq \emptyset$:

   - For each test $t_i \in T \setminus S$:
     - $\text{newCov}_{\text{struct}}(t_i) = R_i \cap R_{\text{uncov}} \cap R_{\text{struct}}$
     - $\text{newCov}_{\text{mut}}(t_i) = R_i \cap R_{\text{uncov}} \cap R_{\text{mut}}$
     - $\text{Score}(t_i) = \alpha \cdot |\text{newCov}_{\text{struct}}(t_i)| + \beta \cdot |\text{newCov}_{\text{mut}}(t_i)|$
   - Choose a test with maximal score:

$$t_{i^*} = \arg\max_{t_i \in T \setminus S} \text{Score}(t_i)$$

     If several tests tie, break ties by the total number of newly covered requirements $|\text{newCov}_{\text{struct}}(t_i)| + |\text{newCov}_{\text{mut}}(t_i)|$.

   - Add t_{i^*} to S.
   - Remove requirements covered by t_{i^*}:

$$R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_{i^*}$$

5. Return S.

Complexity

Let $n = |T|$ be the number of tests and $q = |R|$ the number of requirements (structural + mutants). A straightforward implementation:

- Computes requirement cardinalities and essential tests in $O(nq)$ time (Phase 1).
- In each greedy iteration (Phase 2), scans remaining tests and their coverage sets. If coverage sets are stored as bitsets of length q, each score computation is $O(q)$ and each iteration is $O(nq)$.
- In the worst case, up to n tests are selected, yielding a worst-case time complexity of $O(n^2q)$ for Phase 2.

Overall, MA-EHGS runs in polynomial time in n and q, comparable in practice to other matrix-based greedy heuristics. Efficient bitset implementations and early termination when R_{uncov} becomes small keep runtimes low for the SRM sizes considered in this thesis.

Extended Benchmarks and Reproducibility

This appendix complements the implementation chapter by providing representative empirical results and reproducibility notes for the MA-EHGS implementation within LASSO.

Representative Benchmark Results

Table 1 summarises example experiments on small instructional systems used to validate the implementation. The numbers reflect actual LASSO Arena executions with JaCoCo per-test coverage collection and PIT mutation testing.

Table 1.: Representative MA-EHGS benchmarks from implementation validation.

System	**Original**	**Minimised**	**Reduction**	**Coverage**	**Mutation**
Base64 Encoder	80	8	90.0 %	100 %	100 %
Stack Implementation	18	8	55.6 %	100 %	100 %
Queue Implementation	25	12	52.0 %	100 %	100 %
Parameters: $\alpha = 0.5$, $\beta = 0.5$					

For the Base64 encoder, MA-EHGS selected eight essential tests (distributed between Phase 1 unique coverage selection and Phase 2 weighted greedy selection), achieving a 90% reduction from 80 tests to 8 tests while preserving 100% structural coverage and 100% mutation score. The Stack and Queue implementations similarly demonstrate complete mutation retention. Parameter sensitivity analysis revealed that all α–β combinations (except edge cases $\alpha = 0$ or $\beta = 0$, which yielded 7 tests) produced identical 8-test results for Base64, indicating parameter-insensitive behaviour for this benchmark.

Reproducibility Notes

- **Environment:** LASSO Arena running Docker containers based on `maven:3.9-eclipse-t`
- **Coverage:** JaCoCo 0.8.11 with per-test coverage collection, using reset-based isolation between tests.
- **Mutation:** PIT is configured with common mutation operators (e.g. conditionals, increments, arithmetic, negated conditionals, return values).
- **Parameters:** Default MA-EHGS weights are $\alpha = 0.5$ and $\beta = 0.5$. Parameter sensitivity testing across multiple α–β combinations revealed minimal impact on selection outcomes for the Base64 benchmark, with edge cases ($\alpha = 0$ or $\beta = 0$) producing slightly larger minimised suites (7 vs. 8 tests).

SRM Coverage Extraction Specification

This appendix specifies how LASSO's Stimulus–Response Matrices (SRMs) are converted into a requirements matrix suitable for MA-EHGS.

SRM Structure

LASSO stores SRMs in an Apache Ignite distributed cache:

- **Rows (stimuli):** test sequences, method invocations, or API calls.
- **Columns (systems):** executable systems under test.
- **Cells:** structured response objects M_{ij} with execution results for system j responding to stimulus i.
- **Cache keys (CellId):** include `executionId`, `systemId`, `adapterId`, `variantId` (original/mutant), `sheetId` (test or coverage key), and coordinates.
- **Cache values (CellValue):** include execution outputs, coverage elements, or metrics.

Per-Test Coverage Collection

Per-test coverage is collected via test lifecycle hooks:

1. **Before each test t_i:** coverage counters are reset.
2. **During t_i:** JaCoCo instruments bytecode and records line, branch, and method coverage.
3. **After t_i:** coverage data are exported for that test only.

Coverage is stored as cells with a uniform naming scheme, e.g. `sheetId = "COVERAGE_testName`
and a type field indicating whether the value is a line, branch, or method identifier. This design is language-agnostic and can be mirrored for non-Java systems using other coverage tools.

Mutation Result Collection

For mutation testing:

- Each test t_i is executed against the original system and each mutant variant.
- Results are stored with `sheetId = "testName()"`, and `variantId` distinguishing original vs. individual mutants.
- A mutant is considered *killed* by t_i if any SRM cell for t_i and that mutant differs from the original system's output in the corresponding cell.

Requirements Matrix Construction

The SRM data are then transformed into a unified requirements matrix:

- Tests $T = \{t_1, \ldots, t_n\}$ are derived from distinct test names.
- Structural requirements R_{struct} are derived from covered lines, branches, and methods across all tests and systems.
- Mutation requirements R_{mut} are derived from the set of mutant identifiers killed by at least one test and selected for preservation.
- For each test t_i and requirement r_j, $M_{ij} = 1$ if t_i covers r_j (structural) or kills r_j (mutant). Otherwise $M_{ij} = 0$.

The resulting matrix M is the input to MA-EHGS and to other SRM-based minimisation algorithms.

Licensing Header Template

For completeness, the standard source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk